



เอกสารการออกแบบสถาปัตยกรรมซอฟต์แวร์
โครงการ Smart Niti Application ระบบจัดการนิติบุคคล

จัดทำโดย

นางสาวธนวรรณ ผ่องแผ้ว 6610685171
นางสาวเนตรชนก ยืนดี 6610685221
นายปณณวัฒน์ น้ำคำ 6610685247
นายสิริณัฐ พิมาพิจารณ์ 6610685353
นางสาวอันติมาดา แสงรุ่งเรือง 6610685387

เสนอ

ผศ.ดร.พิศาล แก้วประภา

รายงานนี้เป็นส่วนหนึ่งของหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

สาขาวิชาวิศวกรรมคอมพิวเตอร์

ภาคเรียนที่ 2 ปีการศึกษา 2568

มหาวิทยาลัยธรรมศาสตร์

สารบัญ

1. ภาพรวมของระบบ (System Overview)	2
2. Design Patterns ที่ใช้งานในระบบ	2
2.1 Facade Pattern (Structural Pattern)	3
วัตถุประสงค์	3
การประยุกต์ใช้	3
ไฟล์อ้างอิง	3
ตัวอย่างโค้ด (Code Snippet)	3
2.2 Adapter Pattern (Structural Pattern)	4
วัตถุประสงค์	4
การประยุกต์ใช้	4
ไฟล์อ้างอิง	4
ตัวอย่างโค้ด (Code Snippet)	4
2.3 Singleton Pattern (Creational Pattern)	5
วัตถุประสงค์	5
การประยุกต์ใช้	5
ไฟล์อ้างอิง	5
ตัวอย่างโค้ด (Code Snippet)	5
2.4 Observer Pattern (Behavioral Pattern)	6
วัตถุประสงค์	6
การประยุกต์ใช้	6
ไฟล์อ้างอิง	6
ตัวอย่างโค้ด (Code Snippet)	7
2.5 State Pattern (Behavioral Pattern)	7
วัตถุประสงค์	7
การประยุกต์ใช้	7
ไฟล์อ้างอิง	8
ตัวอย่างโค้ด (Code Snippet)	8
2.6 Factory Method Pattern (Creational Pattern)	8
วัตถุประสงค์	8
การประยุกต์ใช้	8
ไฟล์อ้างอิง	9
ตัวอย่างโค้ด (Code Snippet)	9
3. สถาปัตยกรรมซอฟต์แวร์ (Architectural Pattern)	10
4. สรุปผล (Conclusion)	10

1. ภาพรวมของระบบ (System Overview)

โครงการ Smart Niti Application คือระบบสารสนเทศสำหรับการจัดการงานนิติบุคคลแบบครบวงจร ออกแบบมาเพื่อรองรับการทำงานของทุกฝ่ายที่เกี่ยวข้องในระบบนิติบุคคล ทั้งลูกบ้าน เจ้าหน้าที่นิติบุคคล และช่างซ่อมบำรุง ระบบถูกออกแบบบนสถาปัตยกรรม Client-Server โดยแบ่งการพัฒนาออกเป็น 2 ส่วนหลัก ดังนี้

- 1. **Frontend (Client):** พัฒนาด้วย Flutter (Dart) แบ่งเป็น 3 แอปพลิเคชันย่อย ได้แก่ แอปสำหรับลูกบ้าน (Resident App), แอปสำหรับเจ้าหน้าที่นิติบุคคล (Juristic App) และแอปสำหรับช่างซ่อม (Technician App)
- 2. **Backend (Server):** พัฒนาด้วย FastAPI (Python) ร่วมกับฐานข้อมูล PostgreSQL (Supabase) สำหรับจัดเก็บข้อมูลเชิงสัมพันธ์ และ Supabase Storage สำหรับจัดการไฟล์และรูปภาพ เพื่อให้ระบบมีความยืดหยุ่น บำรุงรักษาง่าย (Maintainability) และรองรับการขยายตัวในอนาคต (Scalability) ทีมพัฒนาได้นำหลักการ Object-Oriented Design (OOD) พร้อมด้วย Design Patterns จำนวน 6 รูปแบบมาประยุกต์ใช้ในการออกแบบโครงสร้างของระบบ

2. Design Patterns ที่ใช้งานในระบบ

จากการวิเคราะห์และออกแบบระบบ ได้มีการนำ Design Patterns มาประยุกต์ใช้ทั้งหมด 6 รูปแบบ ครอบคลุมทั้ง 3 หมวดหมู่หลัก ได้แก่ Creational, Structural และ Behavioral Patterns โดยมีสรุปภาพรวมดังตาราง

ลำดับ	Design Pattern	ประเภท	จุดที่ใช้งาน
1	Facade Pattern	Structural	ResidentFacade, JuristicFacade, TechnicianFacade (Flutter)
2	Adapter Pattern	Structural	EmailAuthAdapter, GoogleAuthAdapter (Flutter), FacebookAuthAdapter(Flutter)
3	Singleton Pattern	Creational	ConnectionManager, SupabaseClient, Database Engine (FastAPI)
4	Observer Pattern	Behavioral	RepairRequestProvider, UserAvatarNotifier (Flutter)
5	State Pattern	Behavioral	TicketStatus Lifecycle (FastAPI)
6	Factory Method	Creational	RepairTicket.fromJson() (Flutter)

2.1 Facade Pattern (Structural Pattern)

วัตถุประสงค์

Facade Pattern มีเป้าหมายเพื่อลดความซับซ้อนของการเรียกใช้งานระบบย่อย (Subsystems) จำนวนมากที่ทำงานอยู่เบื้องหลัง โดยสร้างคลาสตัวกลาง (Facade Class) ทำหน้าที่เป็น "ศูนย์รวมคำสั่ง" (Command center) ที่ฝั่ง Client สามารถเรียกใช้งานได้โดยไม่ต้องรับรู้ถึงรายละเอียดภายในของระบบย่อยนั้น

การประยุกต์ใช้

ในระบบการทำงานจริงของ Smart Niti Application กระบวนการ Login และการจัดการ Session ของผู้ใช้งานเกี่ยวข้องกับ Firebase Authentication โดยตรง แทนที่โค้ดฝั่ง UI จะต้องเรียกใช้ FirebaseAuth ที่ละคำสั่งด้วยตัวเอง ระบบได้แก้ปัญหานี้ด้วยการสร้าง Facade Class ขึ้นมา 3 ตัว ได้แก่:

- ResidentFacade — สำหรับแอปลูกบ้าน
- JuristicFacade — สำหรับแอปเจ้าหน้าที่นิติบุคคล
- TechnicianFacade — สำหรับแอปช่างซ่อม

ตัวอย่างที่ชัดเจนคือกระบวนการ Login ของลูกบ้าน โค้ดฝั่ง UI เพียงเรียก ResidentFacade.login() คำสั่งเดียว โดยไม่จำเป็นต้องรับรู้ถึงกระบวนการเชื่อมต่อกับ Firebase Authentication ที่ซ่อนอยู่เบื้องหลัง ส่งผลให้ลดการ Coupling ระหว่างชั้น UI และ Business Logic ได้อย่างมีประสิทธิภาพ

ไฟล์อ้างอิง

- mobile_app/lib/resident/resident_facade.dart
- technician_app/lib/technician/technician_facade.dart
- juristic_app/lib/features/juristic/juristic_facade.dart

ตัวอย่างโค้ด (Code Snippet)

```
// ไฟล์: mobile_app/lib/resident/resident_facade.dart
import 'package:firebase_auth/firebase_auth.dart';
import '../auth/adapters/auth_adapter.dart';

class ResidentFacade {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  // เมธอด Login ที่ซ่อนความซับซ้อนการทำงานเบื้องหลัง
  Future<User?> login(AuthAdapter adapter) => adapter.login();

  // เมธอด Logout
  Future<void> logout() => _auth.signOut();
}
```

2.2 Adapter Pattern (Structural Pattern)

วัตถุประสงค์

Adapter Pattern ทำหน้าที่เป็น "ตัวแปลง" ที่ช่วยให้ Interface หรือคลาสที่มีรูปแบบไม่เข้ากัน (Incompatible Interfaces) สามารถทำงานร่วมกันได้โดยไม่ต้องแก้ไขโค้ดต้นฉบับของแต่ละฝ่าย

การประยุกต์ใช้

ระบบแอปพลิเคชันรองรับการเข้าสู่ระบบหลายช่องทาง เช่น Email/Password, Google Sign-In และ Facebook Sign-In ซึ่ง SDK ของแต่ละผู้ให้บริการมีชุดคำสั่งที่แตกต่างกัน (Google ใช้ GoogleSignIn, Email ใช้ FirebaseAuth และ Facebook ใช้ FacebookAuth) ทีมพัฒนาแก้ปัญหาโดยการออกแบบ Interface กลางชื่อ AuthAdapter ที่กำหนดว่าทุก Adapter จะต้องมีเมธอด login() และ logout() จากนั้นจึงสร้าง Concrete Adapter ให้แต่ละช่องทาง Login

แนวทางนี้สอดคล้องกับหลักการ Open/Closed Principle (OCP) ใน SOLID เนื่องจากการเพิ่มช่องทาง Login ใหม่ เช่น Apple Sign-In สามารถทำได้โดยสร้าง Adapter ใหม่เพิ่มเติม โดยไม่กระทบโค้ดหลักของแอปพลิเคชัน

ไฟล์อ้างอิง

- mobile_app/lib/auth/adapters/auth_adapter.dart (Target Interface)
- mobile_app/lib/auth/adapters/email_auth_adapter.dart (Concrete Adapter)
- mobile_app/lib/auth/adapters/google_auth_adapter.dart (Concrete Adapter)
- mobile_app/lib/auth/adapters/facebook_auth_adapter.dart (Concrete Adapter)

ตัวอย่างโค้ด (Code Snippet)

```
// 1. Target Interface กลาง
// ไฟล์: mobile_app/lib/auth/adapters/auth_adapter.dart

abstract class AuthAdapter {
  Future<User?> login();
  Future<void> logout();
}

// 2. Concrete Adapter สำหรับการล็อกอินด้วยอีเมล
// ไฟล์: mobile_app/lib/auth/adapters/email_auth_adapter.dart

class EmailAuthAdapter implements AuthAdapter {
  final FirebaseAuth _auth = FirebaseAuth.instance;
  final String email;
  final String password;

  EmailAuthAdapter({required this.email, required this.password});
```

```

@Override
Future<User?> login() async {
  try {
    final credential = await _auth.signInWithEmailAndPassword(
      email: email.trim(),
      password: password.trim(),
    );
    return credential.user;
  } on FirebaseAuthException catch (e) {
    throw Exception(e.message ?? 'Email login failed');
  }
}

@Override
Future<void> logout() async => await _auth.signOut();
}

```

2.3 Singleton Pattern (Creational Pattern)

วัตถุประสงค์

Singleton Pattern รับประกันว่าคลาสใดคลาสหนึ่งจะมีออบเจกต์ (Instance) เพียงตัวเดียวตลอดวงจรการทำงานของโปรแกรม (Global Point of Access) เพื่อให้การจัดการทรัพยากรที่ใช้ร่วมกัน เช่น การเชื่อมต่อฐานข้อมูล เป็นไปอย่างมีระเบียบและประหยัดทรัพยากร

การประยุกต์ใช้

ในฝั่ง Backend ของระบบมีการนำ Singleton Pattern มาใช้ใน 2 บริบทหลัก ดังนี้

- ConnectionManager (WebSocket Manager): ระบบแจ้งเตือนแบบ Real-time ต้องการตัวจัดการสถานะการเชื่อมต่อ (Active Connections) ของผู้ใช้ทุกคนรวมกันในจุดเดียว Backend จึงประกาศ `manager = ConnectionManager()` เป็น Global Instance เพื่อให้ทุก Endpoint สามารถเรียกใช้คำสั่ง `broadcast_json()` เพื่อส่งการแจ้งเตือนหาผู้ใช้ได้
- Database & Storage Clients: การเชื่อมต่อกับฐานข้อมูลผ่าน SQLAlchemy Engine และ Supabase Storage Client ถูกออกแบบเป็น Singleton เพื่อลด Overhead และป้องกัน Connection Overload ที่อาจเกิดจากการสร้าง Connection Object ใหม่ทุกครั้งที่มี HTTP Request เข้ามา

ไฟล์อ้างอิง

- `backend/app/websocket_manager.py`
- `backend/app/core/supabase_client.py`
- `backend/app/database.py`

ตัวอย่างโค้ด (Code Snippet)

```
# ไฟล์: backend/app/websocket_manager.py
```

```

from fastapi import WebSocket

class ConnectionManager:
    def __init__(self):
        self.active_connections: list[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        if websocket in self.active_connections:
            self.active_connections.remove(websocket)

    async def broadcast_json(self, message: dict):
        for connection in self.active_connections:
            await connection.send_json(message)

# สร้าง Global Instance เพียงตัวเดียว (Singleton Instance)
manager = ConnectionManager()

```

2.4 Observer Pattern (Behavioral Pattern)

วัตถุประสงค์

Observer Pattern สร้างความสัมพันธ์แบบ One-to-Many ระหว่างออบเจกต์ เมื่อออบเจกต์ "Subject" มีการเปลี่ยนแปลงสถานะ ออบเจกต์ที่ติดตามอยู่ทั้งหมด (Observers) จะได้รับการแจ้งเตือนและอัปเดตตัวเองโดยอัตโนมัติ โดยที่ Subject ไม่จำเป็นต้องรู้ว่า มี Observer กี่ตัวหรือเป็นตัวแทน

การประยุกต์ใช้

แอปพลิเคชัน Flutter ใช้แนวคิดของ Observer Pattern เป็นกลไกหลักใน State Management โดยใช้คลาส ChangeNotifier และ ValueNotifier ของ Flutter Framework มาเป็น Subject เมื่อผู้ใช้ทำการกรอกข้อมูลในฟอร์มการแจ้งซ่อม หรือเมื่อข้อมูลถูกโหลดจาก API เสร็จสิ้น Provider จะเรียกเมธอด notifyListeners() ซึ่งส่งสัญญาณไปยัง Widget ต่าง ๆ ในหน้าจอ (Observers) ที่ลงทะเบียนรับฟังผ่าน Consumer หรือ Provider.of() ให้ทำการ Rebuild UI ทันทีโดยอัตโนมัติ นอกจากนี้ยังใช้ ValueNotifier สำหรับ sync ข้อมูล Avatar ของผู้ใช้ระหว่างหน้าจอต่างๆ โดยไม่ต้องพึ่ง State Management Library เพิ่มเติม

ไฟล์อ้างอิง

- mobile_app/lib/resident/features/repair_request/provider/repair_request_provider.dart
- mobile_app/lib/resident/core/user_avatar_notifier.dart

ตัวอย่างโค้ด (Code Snippet)

```
// ไฟล์: repair_request_provider.dart

import 'package:flutter/material.dart';

// Subject: สืบทอดจาก ChangeNotifier
class RepairRequestProvider extends ChangeNotifier {
  int _currentStep = 0;
  bool _isLoading = false;

  int get currentStep => _currentStep;
  bool get isLoading => _isLoading;

  void nextStep() {
    if (_currentStep < 2) {
      _currentStep++;
      // แจ้งเตือน Observers (UI Widgets) ให้อัปเดตหน้าจอ
      notifyListeners();
    }
  }

  void reset() {
    _currentStep = 0;
    _isLoading = false;
    notifyListeners();
  }
}
```

2.5 State Pattern (Behavioral Pattern)

วัตถุประสงค์

State Pattern อนุญาตให้ออบเจกต์เปลี่ยนแปลงพฤติกรรม (Behavior) ของมันได้ตามสถานะภายใน (Internal State) ที่เปลี่ยนไป โดยมีเป้าหมายหลักเพื่อการจัดการใช้คำสั่งเงื่อนไข if-else หรือ switch-case ที่ซับซ้อนและยากต่อการบำรุงรักษา แทนที่ด้วยโครงสร้างที่ชัดเจนและขยายได้ง่าย

การประยุกต์ใช้

ตั๋วแจ้งซ่อม (Repair Ticket) มี Lifecycle สถานะที่ชัดเจนและเป็นลำดับขั้น ได้แก่ submitted, assigned, in_progress, done และ cancelled ฝ่าย Backend ได้นำแนวคิด State Pattern มาบังคับข้อจำกัดพฤติกรรมตามสถานะ เช่น:

- หากสถานะเป็น in_progress ระบบจะไม่อนุญาตให้ลูกบ้านเปลี่ยนสถานะเป็น cancelled ได้
- หากสถานะยังไม่ถึง done ระบบจะปฏิเสธ (Reject) การเรียก API เพื่อให้คะแนนช่าง (Rating) ทันที

- การเปลี่ยนสถานะแต่ละขั้นมีการ Validate เส้นทาง (Transition) ที่อนุญาตไว้อย่างชัดเจน

การออกแบบเช่นนี้ทำให้ Lifecycle ของตัวแจ้งซ่อมมีความเป็นระเบียบ ป้องกัน Business Logic ที่ไม่ถูกต้อง และเพิ่มสถานะใหม่ในอนาคตได้โดยไม่กระทบโค้ดที่มีอยู่

ไฟล์อ้างอิง

- backend/app/models/ticket.py (TicketStatus Enum)
- backend/app/api/endpoints/tickets.py (Transition Logic)

ตัวอย่างโค้ด (Code Snippet)

```
# ไฟล์: backend/app/models/ticket.py

import enum
from sqlalchemy import Column, Enum

# ประกาศ States ทั้งหมดของ Ticket
class TicketStatus(str, enum.Enum):
    submitted = 'submitted'
    assigned = 'assigned'
    in_progress = 'in_progress'
    done = 'done'
    cancelled = 'cancelled'

class TicketModel(Base):
    __tablename__ = 'tickets'

    # State field พร้อม Default State เมื่อสร้างตัวใหม่
    status = Column(Enum(TicketStatus), default=TicketStatus.submitted)
```

2.6 Factory Method Pattern (Creational Pattern)

วัตถุประสงค์

Factory Method Pattern ซ่อนรายละเอียดและกระบวนการสร้างออบเจกต์ (Object Instantiation) ไว้ภายในเมธอดพิเศษ ทำให้โค้ดที่ใช้งานออบเจกต์ไม่จำเป็นต้องรับรู้ถึงกระบวนการสร้าง ช่วยเพิ่มความยืดหยุ่นและลดการพึ่งพาระหว่าง Class

การประยุกต์ใช้

ในภาษา Dart/Flutter มีการใช้ Named Constructor ชนิด factory อย่างแพร่หลาย ระบบ Smart Niti ใช้ Factory Method ในการทำ Data Parsing โดยนำข้อมูล JSON ที่รับมาจาก Backend API เข้าไปประมวลผลใน Factory Method ซึ่งทำหน้าที่:

- ตรวจสอบและจัดการค่า null ของแต่ละ Field ด้วยค่า Default
- แปลงชนิดข้อมูล (Type Casting) ให้ถูกต้อง

- ประกอบร่างและคืนค่า Object ที่สมบูรณ์พร้อมใช้งานให้ชั้น UI

การมี Factory Method ทำให้โค้ดชั้น UI สะอาดและไม่ต้องกังวลเรื่องการ Parse ข้อมูลด้วยตัวเอง และหากโครงสร้าง JSON เปลี่ยนแปลงในอนาคต ผู้พัฒนาแก้ไขเพียงจุดเดียวใน Factory Method เท่านั้น

ไฟล์อ้างอิง

- mobile_app/lib/resident/features/repair_history/models/repair_ticket_model.dart
- juristic_app/lib/features/home/models/ticket_model.dart
- technician_app/lib/work_order/models/work.dart
- technician_app/lib/schedule/models/schedule_model.dart

ตัวอย่างโค้ด (Code Snippet)

```
// ไฟล์: repair_ticket_model.dart

class RepairTicket {
  final int id;
  final String reqUserId;
  final String category;
  final String status;

  RepairTicket({
    required this.id,
    required this.reqUserId,
    required this.category,
    required this.status,
  });

  // Factory Method: รับ JSON Map แล้วสร้าง Object ที่สมบูรณ์คืนกลับ
  factory RepairTicket.fromJson(Map<String, dynamic> json) {
    return RepairTicket(
      id: json['id'],
      reqUserId: json['req_user_id'] ?? '',
      category: json['category'] ?? 'other',
      status: json['status'] ?? 'submitted',
    );
  }
}
```

3. สถาปัตยกรรมซอฟต์แวร์ (Architectural Pattern)

นอกจาก Design Patterns ทั้ง 6 รูปแบบแล้ว โครงการ Smart Niti Application ยังได้นำรูปแบบสถาปัตยกรรม MVC-inspired Layered Architecture มาจัดวางโครงสร้างของระบบ เพื่อแบ่งแยกหน้าที่ความรับผิดชอบอย่างชัดเจน (Separation of Concerns) ดังนี้

ชั้น (Layer)	เทคโนโลยี	หน้าที่รับผิดชอบ
Models (Data Layer)	SQLAlchemy / Dart Class	ตัวแทนโครงสร้างฐานข้อมูลและ Data Models ทั้งในฝั่ง Backend (models/ticket.py) และ Frontend (repair_ticket_model.dart) รับผิดชอบการกำหนดรูปแบบข้อมูล
Controllers / Routers (Logic Layer)	FastAPI Endpoints	รับผิดชอบ Business Logic, Validation เงื่อนไข, การเชื่อมต่อ Database และการประมวลผลคำร้องจาก Client ก่อนส่งผลลัพธ์กลับ
Views (Presentation Layer)	Flutter Widgets	จัดการ UI แสดงผล, รับ Input จากผู้ใช้ และเรียกใช้ Facade/Provider เพื่อสื่อสารกับระบบ โดยไม่มี Business Logic ปนอยู่

4. สรุปผล (Conclusion)

การนำ Design Patterns ทั้ง 6 รูปแบบมาประยุกต์ใช้ในโครงการ Smart Niti Application ส่งผลให้ระบบมีคุณสมบัติที่ดีขึ้นในหลายด้าน ดังนี้

- **Modularity:** โค้ดถูกแบ่งออกเป็นโมดูลที่มีหน้าที่ชัดเจน ทำให้ทีมพัฒนาสามารถทำงานบนส่วนต่างๆ ของระบบได้พร้อมกันโดยไม่ขัดแย้งกัน
- **Extensibility:** การเพิ่มฟีเจอร์ใหม่หรือเปลี่ยนบริการที่เชื่อมต่อ (เช่น เพิ่มช่องทาง Login ใหม่) สามารถทำได้โดยไม่ต้องแก้ไขโค้ดหลัก สอดคล้องกับ Open/Closed Principle
- **Maintainability:** โครงสร้างที่ชัดเจนและมีรูปแบบที่เป็นที่รู้จักทำให้นักพัฒนาคนใหม่เข้าใจระบบได้รวดเร็ว และลดเวลาในการ Debug
- **Testability:** การแยก Concerns ออกจากกันอย่างชัดเจนทำให้สามารถเขียน Unit Test สำหรับแต่ละ Module ได้อย่างเป็นอิสระ
- **SOLID Compliance:** การออกแบบครอบคลุมหลักการ SOLID ครบถ้วน โดยเฉพาะ Single Responsibility, Open/Closed และ Dependency Inversion Principles

โดยรวมแล้ว Smart Niti Application แสดงให้เห็นถึงการนำหลักการ Object-Oriented Design มาใช้งานในโปรเจกต์จริงอย่างมีประสิทธิภาพ ระบบมีความพร้อมสำหรับการขยายตัวในเชิงพาณิชย์ และเป็นตัวอย่างของการออกแบบซอฟต์แวร์ที่ดีในอุตสาหกรรม